MULTI-PRODUCT SHOPPING BUDGET CONSTRAINT

Python Programming Project Report

Project topic	Multi-Product Shopping Budget Constraint
Language	Python 3
Purpose	Apply discounts, compare a basket total with a fixed budget, and reduce quantities when necessary

Includes problem statement, objectives, assumptions, algorithm, flowchart, pseudocode, source code, test cases, sample output, and conclusion.

1. Problem Statement

Develop a Python program that evaluates a shopping basket containing multiple products and quantities. The program calculates each product line total, applies a discount rule, compares the discounted basket total with a fixed budget, and determines which quantities should be reduced when the basket exceeds the budget. It then displays the original and adjusted quantities, the final cost, and the remaining budget.

2. Objectives

• Accept a budget and details for multiple products.

• Calculate product subtotals using unit prices and quantities.

• Apply a clearly defined discount condition to eligible products.

• Compare the discounted basket total with the available budget.

• Reduce quantities using a consistent rule until the basket fits the budget or no further reduction is possible.

• Report original quantities, final quantities, reductions, final cost, and remaining budget.

3. Assumptions

• Unit prices and budget are non-negative; quantities are non-negative whole numbers.

• Products priced at Rs 500 or more receive a 10% discount; other products receive no discount.

• Discount eligibility depends only on unit price and does not change when quantity is reduced.

• If the basket exceeds budget, reduce quantities starting with the product having the highest discounted unit price.

• The program never reduces a quantity below zero.

• This is an educational simulation; taxes, delivery charges, coupons, and stock availability are not included.

4. Algorithm

1. Start and input the total budget and number of products.

2. Validate the budget and product count.

3. For each product, read its name, unit price, and quantity; validate the inputs.

4. Set the discount rate to 10% when unit price is at least Rs 500; otherwise set it to 0%.

5. Calculate the original discounted total for each product and the complete basket.

6. If the basket total exceeds the budget, sort products by discounted unit price from highest to lowest.

7. Reduce one unit at a time from each product in that order while the total remains above budget and quantity is available.

8. Recalculate the final basket total and show original quantity, final quantity, and reduced quantity for each product.

9. Display the final total, remaining budget, and whether the basket is within budget.

10. Stop.

5. Flowchart

The flowchart outlines product input, discount calculation, budget checking, and quantity reduction.

START
Input budget and product count
Input each product's price and quantity
Apply discount and calculate basket total
Is total greater than budget?
Yes: reduce highest discounted unit-price quantity first
Recalculate total; repeat while over budget
No / budget met: display adjusted basket
Display final total and remaining budget
END

Flow: Start → Read inputs → Apply discounts → Compare with budget → Reduce quantities if needed → Display adjusted basket → End.

6. Pseudocode

```
BEGIN
    INPUT budget and product_count
    VALIDATE budget and product_count
    products = empty list

    FOR each product:
        INPUT name, unit_price, quantity
        VALIDATE values
        IF unit_price >= 500:
            discount_rate = 10%
        ELSE:
            discount_rate = 0%
        SAVE product and original quantity
    END FOR

    FUNCTION line_total(product):
        RETURN unit_price * quantity * (1 - discount_rate)

    total = SUM of all line totals
    SORT products by discounted unit price descending

    FOR each product in sorted order:
        WHILE total > budget AND quantity > 0:
            reduce quantity by 1
            recalculate total
        END WHILE
    END FOR

    DISPLAY original and final quantities
    DISPLAY final total and budget remaining
END
```

7. Python Source Code

Save as **shopping_budget.py** and run using Python 3.

```
def main():
    print("=== Multi-Product Shopping Budget Constraint ===")

    try:
        budget = float(input("Enter total shopping budget (Rs): "))
        count = int(input("Enter number of products: "))
        if budget < 0 or count <= 0:
            print("Budget must be non-negative and product count must be positive.")
            return
    except ValueError:
        print("Invalid input. Enter a numeric budget and whole-number product count.")
        return

    products = []
    for i in range(count):
        print(f"\\nProduct {i + 1}")
        name = input("Product name: ").strip()
        try:
            price = float(input("Unit price (Rs): "))
            quantity = int(input("Quantity: "))
            if price < 0 or quantity < 0:
                print("Price and quantity cannot be negative. Product skipped.")
                continue
        except ValueError:
            print("Invalid price or quantity. Product skipped.")
            continue

        # Discount rule: 10% off when unit price is Rs 500 or more.
        discount_rate = 0.10 if price >= 500 else 0.0
        products.append({
            "name": name,
            "price": price,
            "quantity": quantity,
            "original_quantity": quantity,
            "discount_rate": discount_rate
        })

    def line_total(product):
        subtotal = product["price"] * product["quantity"]
        return subtotal * (1 - product["discount_rate"])

    def basket_total():
        return sum(line_total(p) for p in products)

    original_total = basket_total()
    print("\\n--- Original Basket ---")
    for p in products:
        print(f"{p['name']}: qty={p['quantity']}, "
              f"unit=Rs {p['price']:.2f}, discount={p['discount_rate']:.0%}, "
              f"line total=Rs {line_total(p):.2f}")
    print(f"Original total after discounts: Rs {original_total:.2f}")
    print(f"Budget: Rs {budget:.2f}")

    # Reduction policy: reduce quantities of the most expensive unit first.
    # Discount eligibility depends only on unit price, so it stays stable
    # when quantity is reduced.
    reduction_order = sorted(
        products, key=lambda p: p["price"] * (1 - p["discount_rate"]), reverse=True
    )

    for p in reduction_order:
        while basket_total() > budget and p["quantity"] > 0:
            p["quantity"] -= 1

    final_total = basket_total()

    print("\\n--- Adjusted Basket ---")
    for p in products:
        reduced = p["original_quantity"] - p["quantity"]
        print(f"{p['name']}: original qty={p['original_quantity']}, "
              f"final qty={p['quantity']}, reduced={reduced}, "
              f"line total=Rs {line_total(p):.2f}")
    print(f"Final total after discounts: Rs {final_total:.2f}")
    print(f"Budget remaining: Rs {budget - final_total:.2f}")

    if final_total <= budget:
        print("Status: Basket is within budget.")
    else:
        print("Status: Budget cannot be met with the available quantities.")
if __name__ == "__main__":
```

```
main()
```

8. Test Cases

ID	Scenario	Expected result
TC1	Basket total is below budget	No quantities reduced; status within budget
TC2	Basket exceeds budget; one expensive product can be reduced	Reduce highest discounted unit-price product first
TC3	Product unit price = Rs 500	10% discount applied
TC4	Product unit price = Rs 499	No discount applied
TC5	Quantity is zero	Line total is zero
TC6	Budget = Rs 0	Reduce all available quantities; final total should be zero
TC7	All quantities reduced but total still over budget	Report budget cannot be met with available quantities
TC8	Negative price or quantity	Product is skipped with a validation message
TC9	Non-numeric price or quantity	Product is skipped with a validation message
TC10	Multiple products	Line totals, discounts, reductions, and final budget are reported

9. Sample Output (Console Screenshot Mock-up)

This is a sample console transcript, not a screenshot captured from a live terminal. Run the code to reproduce it. If your instructor requires an actual screenshot, capture the terminal after running the program.

```
=== Multi-Product Shopping Budget Constraint ===
Enter total shopping budget (Rs): 3000
Enter number of products: 3

Product 1
Product name: Rice Bag
Unit price (Rs): 1200
Quantity: 2

Product 2
Product name: Cooking Oil
Unit price (Rs): 650
Quantity: 2

Product 3
Product name: Biscuits
Unit price (Rs): 100
Quantity: 5

--- Original Basket ---
Rice Bag: qty=2, unit=Rs 1200.00, discount=10%, line total=Rs 2160.00
Cooking Oil: qty=2, unit=Rs 650.00, discount=10%, line total=Rs 1170.00
Biscuits: qty=5, unit=Rs 100.00, discount=0%, line total=Rs 500.00
Original total after discounts: Rs 3830.00
Budget: Rs 3000.00

--- Adjusted Basket ---
Rice Bag: original qty=2, final qty=1, reduced=1, line total=Rs 1080.00
Cooking Oil: original qty=2, final qty=2, reduced=0, line total=Rs 1170.00
Biscuits: original qty=5, final qty=5, reduced=0, line total=Rs 500.00
Final total after discounts: Rs 2750.00
Budget remaining: Rs 250.00
Status: Basket is within budget.
```

10. Conclusion

The Multi-Product Shopping Budget Constraint program calculates shopping costs, applies a defined discount rule, and checks the basket against a fixed budget. When the total is too high, it reduces quantities in a consistent order and reports exactly which purchases were reduced. The project demonstrates loops, lists, sorting, conditional logic, functions, and arithmetic in Python. It can be extended to support tax, shipping, multiple discount types, user priorities, or a graphical shopping interface.

How to run the program

- Save the source code as shopping_budget.py.
- Open a terminal or Command Prompt in the saved file's folder.
- Run: python shopping_budget.py
- Enter the budget, product count, and each product's name, unit price, and quantity.